

DD_MED-01 - Usage Mediation - Developer Implementation Guide

Version: 1.0 Status: Draft for review Bundle: 05_billing Audience: mid-level backend developers, mediation developers, QA
Parent DD: DD_MED-01 - Usage Mediation

1. How To Read This Guide

This guide follows the execution flow from source configuration to raw import, mediation, quarantine repair, and handoff to rating.

MED-01 does not rate usage and does not debit wallets. It produces clean usage records for RAT-01.

2. Step 1 - Configure Usage Source

Create a usage source for VoipSwitch Kenya.

```
insert into usage_source (  
  usage_source_id,  
  operator_code,  
  source_code,  
  source_type,  
  ingestion_mode,  
  status,  
  timezone,  
  config_json,  
  secret_ref,  
  created_at,  
  updated_at  
) values (  
  'usrc-voipswitch-ke',  
  'WIK',  
  'VOIPSWITCH_KE',  
  'VOIPSWITCH',  
  'SFTP_FILE',  
  'ACTIVE',  
  'Africa/Nairobi',  
  '{"filePattern":"cdr_*.csv","delimiter":",",""}',  
  'secret/med/voipswitch-ke',  
  now(),  
  now()  
);
```

Meaning:

- ingestion_mode tells the importer whether to pull files, call an API, receive pushes, or accept manual upload.
- timezone is used when source timestamps do not include an offset.
- secret_ref points to SFTP/API credentials.

3. Step 2 - Source Sends Or Exposes CDR File

Example VoipSwitch-style CSV:

```
cdr_id,caller,callee,start_time,end_time,duration_sec,disposition,sip_account  
VS-CDR-998877,0700111222,0711222333,2026-06-02 10:15:00,2026-06-02 10:16:15,75,ANSWERED,voice-5012  
VS-CDR-998878,0700111222,999,2026-06-02 10:20:00,2026-06-02 10:20:30,30,ANSWERED,voice-5012
```

The importer stores the original file in object storage before parsing rows.

4. Step 3 - Create Raw Batch

Create one batch row for the file/API window.

```
insert into raw_usage_batch (  
  batch_id,
```

```

usage_source_id,
operator_code,
source_batch_ref,
period_start,
period_end,
status,
source_record_count,
source_total_duration_seconds,
stored_record_count,
mediated_record_count,
quarantine_record_count,
file_checksum,
created_at,
updated_at
) values (
'rub-20260602-1000',
'usrc-voipswitch-ke',
'WIK',
'cdr_20260602_1000.csv',
'2026-06-02T10:00:00+03:00',
'2026-06-02T10:59:59+03:00',
'IMPORTING',
2,
105,
0,
0,
0,
'sha256:abc123',
now(),
now()
);

```

If a file with the same source and checksum was already imported, reject the import as duplicate unless an admin explicitly starts a reprocess.

5. Step 4 - Store Raw CDR Rows

For each parsed row, build a deterministic record_hash.

Recommended hash input for VoipSwitch:

```
source_code + cdr_id
```

If source CDR id is missing:

```
source_code + caller + callee + start_time + duration_sec + sip_account
```

Insert raw row:

```

insert into raw_usage_record (
raw_record_id,
batch_id,
usage_source_id,
operator_code,
source_record_id,
source_sequence_no,
record_hash,
raw_payload_json,
processing_status,
created_at
) values (
'rur-20260602-000001',
'rub-20260602-1000',
'usrc-voipswitch-ke',
'WIK',
'VS-CDR-998877',
1,
'sha256:def456',
'{"cdr_id":"VS-CDR-998877","caller":"0700111222","callee":"0711222333","start_time":"2026-06-02 10:15:00","duration_sec":10}',
'NEW',
now()
);

```

Do not normalize first and store later. Raw row storage comes first.

6. Step 5 - Normalize Raw CDR

The mediation worker loads unprocessed rows:

```
select *
from raw_usage_record
where processing_status = 'NEW'
order by created_at asc
limit 500;
```

For each row:

1. Parse source payload by source type.
2. Normalize caller and callee.
3. Parse timestamps using source timezone.
4. Validate duration.
5. Resolve subscription/customer/service.
6. Create mediated record or quarantine.

Pseudo-code:

```
public function mediateRawRecord(RawUsageRecord $raw): void
{
    $source = $this->sources->find($raw->usage_source_id);
    $parsed = $this->parserRegistry->for($source->source_type)->parse($raw->raw_payload_json);

    $normalized = $this->normalizer->normalizeVoiceCdr($source, $parsed);
    $validation = $this->validator->validate($normalized);

    if (!$validation->ok()) {
        $this->quarantine($raw, $validation->reasonCode(), $validation->details());
        return;
    }

    $mapping = $this->subscriptionResolver->resolveBySipAccountOrNumber(
        $raw->operator_code,
        $normalized->sipAccount,
        $normalized->callingNumberNormalized,
        $normalized->callStartedAt
    );

    if (!$mapping) {
        $this->quarantine($raw, 'SUBSCRIPTION_NOT_FOUND', $normalized->toArray());
        return;
    }

    $this->createMediatedUsage($raw, $normalized, $mapping);
}
```

7. Step 6 - Normalize Numbers

For WIK Kenya, local mobile number:

```
0700111222 -> +254700111222
0711222333 -> +254711222333
```

Emergency numbers remain short:

```
999 -> 999
112 -> 112
```

Implementation rule:

- Keep national short codes unchanged if configured as short-code prefixes.
- Convert local MSISDNs to E.164.
- Strip spaces, dashes, and brackets.
- Reject impossible numbers into quarantine.

8. Step 7 - Resolve Subscription

Resolution inputs:

Input	Purpose
sip_account	Best lookup key for VoIP service.
calling number	Fallback DID lookup.
call start time	Historical lookup if DID moved between subscriptions.
operator code	Tenant boundary.

Expected mapping result:

```
{
  "subscriptionId": "SUB-700045",
  "customerId": "CUS-100045",
  "accountId": "ACC-900045",
  "serviceRef": "svc_VOIP_STD",
  "packageRef": "pkg_INET_VOICE_STD"
}
```

The lookup can be implemented through an ILM/SUB read API or local mapping table, but it must support historical lookup by call time.

9. Step 8 - Create Mediated Usage

Insert:

```
insert into mediated_usage_record (
  mediated_usage_id,
  raw_record_id,
  operator_code,
  usage_type,
  subscription_id,
  customer_id,
  account_id,
  service_ref,
  package_ref,
  calling_number_normalized,
  called_number_normalized,
  call_direction,
  call_started_at,
  call_ended_at,
  duration_seconds,
  disposition,
  mediation_status,
  created_at
) values (
  'med-20260602-000001',
  'rur-20260602-000001',
  'WIK',
  'VOICE_CALL',
  'SUB-700045',
  'CUS-100045',
  'ACC-900045',
  'svc_VOIP_STD',
  'pkg_INET_VOICE_STD',
  '+254700111222',
  '+254711222333',
  'OUTBOUND',
  '2026-06-02T10:15:00+03:00',
  '2026-06-02T10:16:15+03:00',
  75,
  'ANSWERED',
  'READY_FOR_RATING',
  now()
);
```

Update raw row:

```
update raw_usage_record
set processing_status = 'MEDIATED',
    processed_at = now()
where raw_record_id = 'rur-20260602-000001';
```

Emit:

```
{
  "eventType": "UsageMediated",
  "operatorCode": "WIK",
  "mediatedUsageId": "med-20260602-000001",
}
```

```

    "usageType": "VOICE_CALL",
    "subscriptionId": "SUB-700045",
    "customerId": "CUS-100045",
    "serviceRef": "svc_VOIP_STD",
    "packageRef": "pkg_INET_VOICE_STD",
    "calledNumberNormalized": "+254711222333",
    "callDirection": "OUTBOUND",
    "callStartedAt": "2026-06-02T10:15:00+03:00",
    "durationSeconds": 75
  }
}

```

RAT-01 consumes this event or polls READY_FOR_RATING records.

10. Step 9 - Handle Duplicates

If inserting raw row violates unique (usage_source_id, record_hash), load the existing row.

If raw payload is identical:

```

update raw_usage_record
set processing_status = 'DUPLICATE',
    processed_at = now()
where raw_record_id = 'new-row-id';

```

Do not create another mediated record.

If hash matches but payload differs, quarantine:

```
reason_code = DUPLICATE_CONFLICT
```

This indicates source data changed for the same CDR identity.

11. Step 10 - Quarantine Invalid Record

Example: subscription cannot be resolved.

```

insert into mediation_quarantine (
  quarantine_id,
  raw_record_id,
  operator_code,
  reason_code,
  severity,
  status,
  created_at
) values (
  'mq-20260602-000001',
  'rur-20260602-000002',
  'WIK',
  'SUBSCRIPTION_NOT_FOUND',
  'HIGH',
  'OPEN',
  now()
);

```

Update raw row:

```

update raw_usage_record
set processing_status = 'QUARANTINED',
    processed_at = now()
where raw_record_id = 'rur-20260602-000002';

```

Emit:

```

{
  "eventType": "UsageQuarantined",
  "operatorCode": "WIK",
  "rawRecordId": "rur-20260602-000002",
  "reasonCode": "SUBSCRIPTION_NOT_FOUND",
  "severity": "HIGH"
}

```

12. Step 11 - Repair And Replay Quarantine

Backoffice calls:

```
POST /api/usage-mediation/quarantine/mq-20260602-000001/repair-and-replay
Content-Type: application/json

{
  "subscriptionId": "SUB-700045",
  "customerId": "CUS-100045",
  "serviceRef": "svc_VOIP_STD",
  "packageRef": "pkg_INET_VOICE_STD",
  "resolutionNotes": "Mapped legacy SIP account voice-5012 to active subscription."
}
```

MED-01 must:

1. Store repair payload and user id.
2. Re-run validation using repair values.
3. Create mediated usage if valid.
4. Mark quarantine RESOLVED.
5. Emit UsageMediationReplayed.

Do not overwrite the original raw payload.

13. Step 12 - Mark Rating Handoff

When RAT-01 accepts the mediated record, update:

```
POST /api/usage-mediation/records/med-20260602-000001/mark-sent-to-rating

{
  "ratingRequestId": "ratreq-001"
}
```

After rating succeeds:

```
POST /api/usage-mediation/records/med-20260602-000001/mark-rated

{
  "ratedUsageId": "rated-20260602-000001"
}
```

This lets Backoffice trace a raw CDR to mediated record to rated usage.

14. Step 13 - Reconcile Batch

After mediation completes, run:

```
med:reconcile-batch --batch=rub-20260602-1000
```

Create reconciliation run:

```
insert into mediation_reconciliation_run (
  run_id,
  operator_code,
  usage_source_id,
  batch_id,
  period_start,
  period_end,
  status,
  source_count,
  stored_count,
  mediated_count,
  quarantine_count,
  mismatch_count,
  started_at
) values (
  'mrr-20260602-000001',
  'WIK',
  'usrc-voipswitch-ke',
  'rub-20260602-1000',
  '2026-06-02T10:00:00+03:00',
  '2026-06-02T10:59:59+03:00',
  'RUNNING',
  2,
  2,
  1,
  null,
  null
)
```

```

1,
0,
now()
);

```

Check:

```

source_count == stored_count
stored_count == mediated_count + quarantine_count + duplicate_count
source_total_duration_seconds roughly equals sum(raw duration) unless source excludes failed calls

```

If mismatched, mark run PARTIAL and show on Backoffice dashboard.

15. Suggested Worker Commands

These are Laravel commands or queue jobs and can be consolidated into shared billing workers.

```

med:pull-source-files --source=VOIPSWITCH_KE
med:import-batch --batch=rub-20260602-1000
med:process-raw-records
med:reconcile-batch --batch=rub-20260602-1000
med:replay-quarantine --quarantine=mq-20260602-000001
med:close-stale-batches

```

Recommended schedule:

Command	Frequency
med:pull-source-files	Every 5-15 minutes, depending source availability.
med:process-raw-records	Continuous queue worker or every minute.
med:reconcile-batch	After batch processing, plus nightly.
med:close-stale-batches	Hourly.
med:replay-quarantine	Manual or event-driven after repair.

16. Error Codes

Error code	Meaning
MED_SOURCE_INACTIVE	Usage source is not active.
MED_BATCH_DUPLICATE	Same source file/checksum already imported.
MED_PARSE_FAILED	Source row cannot be parsed.
MED_REQUIRED_FIELD_MISSING	Required CDR field missing.
MED_INVALID_TIMESTAMP	Timestamp cannot be parsed.
MED_INVALID_DURATION	Duration is negative or inconsistent.
MED_NUMBER_NORMALIZATION_FAILED	Caller/callee cannot be normalized.
MED_SUBSCRIPTION_NOT_FOUND	No billable subscription mapping found.
MED_DUPLICATE_CONFLICT	Same CDR identity with conflicting payload.
MED_UNSUPPORTED_DISPOSITION	Source disposition should not be mediated.

17. Minimum Implementation Order

Build in this order:

1. Migrations for source, batch, raw record, mediated record, quarantine, reconciliation run.
2. Usage source CRUD API.
3. File/manual upload importer.
4. VoipSwitch CSV parser.
5. Record hash builder and duplicate handling.
6. Number normalizer for Kenya.

7. Subscription resolver interface.
8. Mediation worker.
9. UsageMediated and UsageQuarantined events.
10. Quarantine APIs.
11. Repair and replay.
12. Batch reconciliation.
13. Backoffice read APIs.
14. RAT-01 handoff status updates.

18. Test Scenarios

18.1 Valid Answered CDR

Input:

- Caller 0700111222.
- Callee 0711222333.
- Duration 75.
- Subscription mapping exists.

Expected:

- Raw row stored.
- Mediated record created.
- Event UsageMediated.

18.2 Duplicate CDR

Input:

- Same cdr_id imported twice.

Expected:

- One mediated record.
- Duplicate raw row marked DUPLICATE or duplicate import rejected.

18.3 Duplicate Conflict

Input:

- Same cdr_id, different duration.

Expected:

- Quarantine with DUPLICATE_CONFLICT.
- No second mediated record.

18.4 Unknown Subscription

Input:

- SIP account not mapped.

Expected:

- Quarantine SUBSCRIPTION_NOT_FOUND.
- Repair can replay with subscription id.

18.5 Failed Call

Input:

- Disposition NO_ANSWER.

Expected:

- Not sent to rating by default.
- Either ignored with audit or quarantined based on operator policy.

18.6 Batch Reconciliation

Input:

- Source file count 100.
- Stored 100, mediated 95, quarantined 5.

Expected:

- Reconciliation completed with mismatch count 0.

19. Developer Checklist

Before marking MED-01 complete:

- Raw records are stored before processing.
- Duplicate handling prevents double mediation.
- Number normalization is deterministic.
- Subscription resolver supports historical lookup.
- Quarantine preserves original raw payload.
- Repair stores user id and notes.
- Mediated record has enough fields for RAT-01.
- UsageMediated event is emitted once per mediated record.
- Batch reconciliation is visible to Backoffice.
- MED-01 does not calculate monetary charges.